

THE FUJISAKI-OKAMOTO TRANSFORM

ADVANCED TOPICS IN ~~CYBERSECURITY~~ CRYPTOGRAPHY (7CCSMATC)

Martin R. Albrecht

MAIN REFERENCE

Eiichiro Fujisaki and Tatsuaki Okamoto.
Secure Integration of Asymmetric and Symmetric Encryption Schemes. In:
Journal of Cryptology 26.1 (Jan. 2013),
pp. 80–101. DOI:
10.1007/s00145-011-9114-1



- Alexander W. Dent. **A Designer's Guide to KEMs.** In: *9th IMA International Conference on Cryptography and Coding*. Ed. by Kenneth G. Paterson. Vol. 2898. LNCS. Springer, Berlin, Heidelberg, Dec. 2003, pp. 133–151. DOI: 10.1007/978-3-540-40974-8_12
- Martin R. Albrecht, Emmanuela Orsini, Kenneth G. Paterson, Guy Peer, and Nigel P. Smart. **Tightly Secure Ring-LWE Based Key Encapsulation with Short Ciphertexts.** In: *ESORICS 2017, Part I*. ed. by Simon N. Foley, Dieter Gollmann, and Einar Sneekkenes. Vol. 10492. LNCS. Springer, Cham, Sept. 2017, pp. 29–46. DOI: 10.1007/978-3-319-66402-6_4
- Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. **A Modular Analysis of the Fujisaki-Okamoto Transformation.** In: *TCC 2017, Part I*. ed. by Yael Kalai and Leonid Reyzin. Vol. 10677. LNCS. Springer, Cham, Nov. 2017, pp. 341–371. DOI: 10.1007/978-3-319-70500-2_12

PUBLIC-KEY ENCRYPTION (PKE) WITH EXPLICIT RANDOMNESS

A Public-Key Encryption (PKE) scheme is a triple of PPT algorithms (KeyGen, Enc, Dec) with the following syntax and operation:

- KeyGen** The key generation algorithm is a randomised algorithm taking as input a security parameter 1^λ and outputs a public/secret key-pair (pk, sk), the **public key** and the **secret key** respectively.
- Enc** The encryption algorithm is a **deterministic** algorithm taking as input a public-key pk and a message m **and some randomness** r and outputs an encryption of m under pk.
- Dec** The decryption algorithm is a deterministic algorithm taking as input a ciphertext c and a secret-key sk, and outputs a message m (or an error message $\perp$ indicating a decryption failure).

KEY ENCAPSULATION MECHANISM (KEM)

A Key Encapsulation Mechanism (KEM) is a triple of PPT algorithms (KeyGen, Encap, Decap) with the following syntax and operation:

- KeyGen** The key generation algorithm is a randomised algorithm taking as input a security parameter 1^λ and outputs a public/secret key-pair (pk, sk) , the **public key** and the **secret key** respectively.
- Encap** The encapsulation algorithm is a randomised algorithm taking as input a public-key pk and outputs a key $k \in \mathcal{K}$ and an encryption of k under pk .
- Decap** The decapsulation algorithm is a deterministic algorithm taking as input a ciphertext c and a secret-key sk , and outputs a key k (or an error message $\perp$ indicating a decryption failure).

IND-CPA PKE (RECAP)

IND-CPA _{PKE}	$C(m_0, m_1)$
1: $(pk, sk) \leftarrow \$ \text{KeyGen}(1^\lambda)$	1: if $ m_0 \neq m_1 $ then
2: $b \leftarrow \$ \{0, 1\}$	2: return $\perp$
3: $b' \leftarrow \mathcal{D}^c(pk)$	3: $c \leftarrow \$ \text{Enc}(pk, m_b)$
4: return $b = b'$	4: return c

Figure 1: IND-CPA Security Game (PKE).

$$\text{Adv}_{PKE}^{\text{ind-cpa}}(\mathcal{D}) = \left| \Pr[\text{IND-CPA}^{\mathcal{D}} = 1] - 1/2 \right|$$

IND-CCA KEM

IND-CCA _{KEM}	C()	D(c)
1: $\mathcal{C} \leftarrow \emptyset$	1: $k_0 \leftarrow \$ \mathcal{K}$	1: if $c \in \mathcal{C}$ then
2: $(pk, sk) \leftarrow \$ \text{KeyGen}(1^\lambda)$	2: $c, k_1 \leftarrow \$ \text{Encap}(pk)$	2: return $\perp$
3: $b \leftarrow \$ \{0, 1\}$	3: $\mathcal{C} \leftarrow \mathcal{C} \cup \{c\}$	3: return $\text{Decap}(sk, c)$
4: $b' \leftarrow \mathcal{D}^{c, D}(pk)$	4: return c, k_b	
5: return $b = b'$		

$$\text{Adv}_{KEM}^{\text{ind-cca}}(\mathcal{D}) = |\Pr[\text{IND-CCA}^{\mathcal{D}} = 1] - 1/2|.$$

KeyGen(1^λ)

1: $(pk, sk) \leftarrow \$ \text{PKE.KeyGen}(1^\lambda)$
2: **return** pk, sk

Encap(pk)

1: $x \leftarrow \$ \{0, 1\}^\lambda$
2: $r \leftarrow H(x)$ // RO
3: $c \leftarrow \text{PKE.Enc}(pk, x, r)$
4: $k \leftarrow \text{KDF}(x)$ // different RO
5: **return** c, k

Decap(sk, c)

1: $y \leftarrow \text{PKE.Dec}(sk, c)$
2: $s \leftarrow H(y)$
3: $d \leftarrow \text{PKE.Enc}(pk, y, s)$
4: **if** $d \neq c$ **return** $\perp$
5: $k \leftarrow \text{KDF}(y)$
6: **return** k

- $\text{Encap}(\text{pk})$ is essentially $\text{PKE.Enc}()$ for a random x with the randomness r computed as the output of the random oracle $H(x)$.
- $\text{Decap}(\text{sk}, c)$ is essentially $\text{PKE.Dec}()$ but with an additional check after, called the “re-encryption check”.
 - It uses the fact that if we know x then we also know r that was used to call $\text{PKE.Enc}(\text{pk}, x, r)$ so we can “recompute” and check if we get c again.
 - This seems pointless for now, but it will be critical in the proof.

ROs all the way down

It is critical to keep in mind that in the above construction, we have used two **random oracles**: $H()$ and $\text{KDF}()$.

IND-CCA KEM: PROOF IDEA I

- We will show that if there is an adversary $\mathcal{D}$ that can break our CCA-secure KEM in the IND-CCA security game then there is also an adversary $\mathcal{A}$ that breaks our CPA-secure PKE in the IND-CPA game.
- Then, if we assume that our IND-CPA PKE is secure, i.e. that there is no $\mathcal{A}$ in the IND-CPA game, this implies that our IND-CCA KEM is secure in the IND-CCA game, i.e. there is no $\mathcal{D}$.

Punchline

$\mathcal{D}$ implies $\mathcal{A}$, but $\mathcal{A}$ does not exist $\Rightarrow \mathcal{D}$ does not exist.

IND-CCA KEM: PROOF IDEA II

We will build $\mathcal{A}$ from $\mathcal{D}$.

- We're going to put $\mathcal{D}$ in a box, where it will start interacting with what it believes to be the IND-CCA secure KEM.
- But we simulate all aspects of this little world such that when $\mathcal{D}$ succeeds we can use this to break the IND-CPA PKE in the IND-CPA game.
- The problem is that $\mathcal{D}$ and $\mathcal{A}$ live in different worlds: IND-CCA (KEM) game and IND-CPA (PKE) game.
- The key problem is that $\mathcal{D}$ expects a decapsulation oracle, it is a CCA attacker.
- But in the world where we are building $\mathcal{A}$ (the IND-CPA game) this decapsulation oracle does not exist: all we have is
 - our challenge ciphertext c ,
 - two messages m_0 and m_1 (that we can choose) and
 - the ability to encrypt whatever we want using the provided public key pk .
- We need to simulate the decapsulation oracle for $\mathcal{D}$ somehow.

- We could attempt to always return garbage or a random plaintext whenever $\mathcal{D}$ calls it.
- But $\mathcal{D}$ could easily detect this deception:
 - It prepares a (ciphertext, key) pair as $c, k = \text{Encap}(\text{pk})$ and calls the decapsulation oracle on c . If it does not get k back, it knows we are cheating.
- So we must implement the decapsulation oracle correctly without having access to the secret key for decapsulation
 - We are trying to build $\mathcal{A}$ which is pointless if we already know the secret key.
- To accomplish this feat we make use of the random oracles.

The Trick

The hash functions $H()$ and $\text{KDF}()$ that $\mathcal{D}$ calls are oracles: **we** provide them to $\mathcal{D}$!

Game ₀	C()	D(c)
1: $\mathcal{C} \leftarrow \emptyset$	1: $k_0 \leftarrow \$ \mathcal{K}$	1: if $c \in \mathcal{C}$ then
2: $\text{pk}, \text{sk} \leftarrow \$ \text{KeyGen}(1^\lambda)$	2: $c, k_1 \leftarrow \$ \text{Encap}(\text{pk})$	2: return $\perp$
3: $b \leftarrow \$ \{0, 1\}$	3: $\mathcal{C} \leftarrow \mathcal{C} \cup \{c\}$	3: return $\text{Decap}(\text{sk}, c)$
4: $b' \leftarrow \mathcal{D}^{\mathcal{C}, \text{D}, \text{H}, \text{K}}(\text{pk})$	4: return c, k_b	
5: return $b = b'$	K(x)	
H(x)	1: return $\text{KDF}(x)$	
1: return H(x)		

Game ₁	C()	D(c)
1: $\mathcal{C} \leftarrow \emptyset$	1: $x \leftarrow \{0, 1\}^\lambda$	1: if $c \in \mathcal{C}$ then
2: $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$	2: $r \leftarrow H(x)$	2: return $\perp$
3: $b \leftarrow \{0, 1\}$	3: $c \leftarrow \text{PKE.Enc}(pk, x, r)$	3: $y \leftarrow \text{PKE.Dec}(sk, c)$
4: $b' \leftarrow \mathcal{D}^{\mathcal{C}, D, H, K}(pk)$	4: $\mathcal{C} \leftarrow \mathcal{C} \cup \{c\}$	4: $s \leftarrow H(y)$
5: return $b = b'$	5: $k_0 \leftarrow \mathcal{K}; k_1 \leftarrow K(x)$	5: $d \leftarrow \text{PKE.Enc}(pk, y, s)$
<u>H(x)</u>	6: return c, k_b	6: if $d \neq c$ return $\perp$
1: return H(x)	<u>K(x)</u>	7: $k \leftarrow K(y)$
	1: return KDF(x)	8: return k

Game ₂	C()	D(c)
1: $\mathcal{C}, \mathcal{T}_H, \mathcal{T}_K \leftarrow \emptyset, \emptyset, \emptyset$ 2: $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ 3: $b \leftarrow \{0, 1\}$ 4: $b' \leftarrow \mathcal{D}^{C,D,H,K}(pk)$ 5: return $b = b'$	1: $x \leftarrow \{0, 1\}^\lambda$ 2: $r \leftarrow H(x)$ 3: $c \leftarrow \text{PKE.Enc}(pk, x, r)$ 4: $\mathcal{C} \leftarrow \mathcal{C} \cup \{c\}$ 5: $k_0 \leftarrow \mathcal{K}; k_1 \leftarrow K(x)$ 6: return c, k_b	1: if $c \in \mathcal{C}$ then 2: return $\perp$ 3: $y \leftarrow \text{PKE.Dec}(sk, c)$ 4: $s \leftarrow H(y)$ 5: $d \leftarrow \text{PKE.Enc}(pk, y, s)$ 6: if $d \neq c$ return $\perp$ 7: $k \leftarrow K(y)$ 8: return k
H(x) 1: if $x \notin \mathcal{T}_H.\text{keys}$ 2: $\mathcal{T}_H[x] \leftarrow \{0, 1\}^\lambda$ 3: return $\mathcal{T}_H[x]$	K(x) 1: if $x \notin \mathcal{T}_K.\text{keys}$ 2: $\mathcal{T}_K[x] \leftarrow \{0, 1\}^\lambda$ 3: return $\mathcal{T}_K[x]$	

Game₃

```

1:  $\mathcal{C}, \mathcal{T}_H, \mathcal{T}_K, \mathcal{T}_c \leftarrow \emptyset, \emptyset, \emptyset, \emptyset$ 
2:  $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ 
3:  $b \leftarrow \{0, 1\}$ 
4:  $b' \leftarrow \mathcal{D}^{\mathcal{C}, \mathcal{D}, H, K}(pk)$ 
5: return  $b = b'$ 

```

H(x)

```

1: if  $x \notin \mathcal{T}_H.\text{keys}$ 
2:    $\mathcal{T}_H[x] \leftarrow \{0, 1\}^\lambda$ 
3:    $\mathcal{T}_c[\text{PKE.Enc}(pk, x, \mathcal{T}_H[x])] \leftarrow x$ 
4: return  $\mathcal{T}_H[x]$ 

```

C()

```

1:  $x \leftarrow \{0, 1\}^\lambda$ 
2:  $r \leftarrow H(x)$ 
3:  $c \leftarrow \text{PKE.Enc}(pk, x, r)$ 
4:  $\mathcal{C} \leftarrow \mathcal{C} \cup \{c\}$ 
5:  $k_0 \leftarrow \mathcal{K}; k_1 \leftarrow K(x)$ 
6: return  $c, k_b$ 

```

K(x)

```

1: if  $x \notin \mathcal{T}_K.\text{keys}$ 
2:    $\mathcal{T}_K[x] \leftarrow \{0, 1\}^\lambda$ 
3:    $\mathcal{T}_c[\text{PKE.Enc}(pk, x, H(x))] \leftarrow x$ 
4: return  $\mathcal{T}_K[x]$ 

```

D(c)

```

1: if  $c \in \mathcal{C}$  then
2:   return  $\perp$ 
3:  $y \leftarrow \text{PKE.Dec}(sk, c)$ 
4:  $s \leftarrow H(y)$ 
5:  $d \leftarrow \text{PKE.Enc}(pk, y, s)$ 
6: if  $d \neq c$  return  $\perp$ 
7:  $k \leftarrow K(y)$ 
8: return  $k$ 

```

Game₄

```

1:  $\mathcal{C}, \mathcal{T}_H, \mathcal{T}_K, \mathcal{T}_c \leftarrow \emptyset, \emptyset, \emptyset, \emptyset$ 
2:  $(pk, sk) \leftarrow \text{KeyGen}(1^\lambda)$ 
3:  $b \leftarrow \{0, 1\}$ 
4:  $b' \leftarrow \mathcal{D}^{E, D, H, K}(pk)$ 
5: return  $b = b'$ 

```

H(x)

```

1: if  $x \notin \mathcal{T}_H.\text{keys}$ 
2:    $\mathcal{T}_H[x] \leftarrow \{0, 1\}^\lambda$ 
3:    $c \leftarrow \text{PKE}.\text{Enc}(pk, x, \mathcal{T}_H[x])$ 
4:    $\mathcal{T}_c[c] \leftarrow x$ 
5: return  $\mathcal{T}_H[x]$ 

```

C()

```

1:  $x \leftarrow \{0, 1\}^\lambda$ 
2:  $r \leftarrow H(x)$ 
3:  $c \leftarrow \text{PKE}.\text{Enc}(pk, x, r)$ 
4:  $\mathcal{C} \leftarrow \mathcal{C} \cup \{c\}$ 
5:  $k_0 \leftarrow \mathcal{K}; k_1 \leftarrow K(x)$ 
6: return  $c, k_b$ 

```

K(x)

```

1: if  $x \notin \mathcal{T}_K.\text{keys}$ 
2:    $\mathcal{T}_K[x] \leftarrow \{0, 1\}^\lambda$ 
3:    $c \leftarrow \text{PKE}.\text{Enc}(pk, x, H(x))$ 
4:    $\mathcal{T}_c[c] \leftarrow x$ 
5: return  $\mathcal{T}_K[x]$ 

```

D(c)

```

1: if  $c \in \mathcal{C}$  then
2:   return  $\perp$ 
3: if  $c \in \mathcal{T}_c.\text{keys}$ 
4:   return  $K(\mathcal{T}_c[c])$ 
5: else return  $\perp$ 
6:

```

ANALYSING THE CHANGES

1. If $\mathcal{D}$ calls $(c, \cdot) \leftarrow \text{Encap}()$ and thus $H(\cdot)$ before calling $D(c)$ then we can simulate decapsulation correctly. All good here.
2. If $\mathcal{D}$ sends anything else then our simulation returns $\perp$, but so does the real $\text{Decap}()$ function with high probability:
 - It runs the re-encryption check to see if c is the output of $\text{PKE.Enc}(pk, x, H(x))$ for the x output by $\text{PKE.Dec}(sk, c)$.

Task Ahead

$\mathcal{D}$ can only detect that we are cheating if it manages to produce such a c without ever having called $H(x)$ for the matching x .

WHAT CAN GO WRONG?

1. $\mathcal{D}$ could have guessed the output of $H(x)$. But since $H(x)$ is a **random** oracle, the probability of that happening is $1/2^\lambda$ per query
2. A different (x, r') pair with $r' \neq H(x)$ produced c , i.e. we have a collision on c .

To bound this, we need an **additional** property of our IND-CPA PKE:

Definition (γ -uniformity)

Let $(\text{KeyGen}, \text{Enc}, \text{Dec})$ be a PKE with $\text{Enc} : \mathcal{M} \times \mathcal{R} \rightarrow \mathcal{C}$ being the encryption function mapping messages and randomness to ciphertexts. PKE is γ -uniform if for all public keys pk output by KeyGen , all $m \in \mathcal{M}$ and all $c \in \mathcal{C}$, we have

$$\Pr[r \in \mathcal{R} : c = \text{Enc}(\text{pk}, m, r)] \leq \gamma.$$

Applying the union bound, we get $\leq \gamma \cdot q_D$ for the probability of our simulation going wrong, where q_D is the number of decapsulation queries made.

- We have managed to simulate a decapsulation oracle without access to the secret key.
- Our CPA to CCA upgrade is almost complete, we only need to spell out what we actually do with this simulation.
- $\mathcal{D}$ will be on an attack rampage, how do we turn that into a successful $\mathcal{A}$?

BREAKING IND-CPA PKE I

- We intend to build $\mathcal{A}$ (a successful IND-CPA PKE adversary) from $\mathcal{D}$ (a successful IND-CCA KEM adversary).
- The IND-CPA PKE game consists of the adversary picking two messages m_0 and m_1 and handing them to the challenger. The challenger will then randomly select one and encrypt it as ciphertext c .
- The adversary must figure out which one it was.
- $\mathcal{A}$ picks two random messages m_0 and m_1 and submit them to the challenger. We get back a challenge ciphertext c^* .
- $\mathcal{D}$ will expect a challenge ciphertext and a key k .
 - It then has to decide if the key k is the one encapsulated under the ciphertext or not.
- Thus, $\mathcal{A}$ picks a random $k^* \leftarrow \{0, 1\}^\lambda$ and passes (c^*, k^*) to $\mathcal{D}$.

- While we have passed something of the right form to $\mathcal{D}$, the ciphertext c^* is never a correct encapsulation of k^* even if $k^* = \text{KDF}(m_b)$:
 - In the IND-CPA game, $\text{PKE.Enc}(pk, m_b, r)$ is called with some randomness r that is independent of the message m_b .
 - In the IND-CCA game we expect $r = H(m_b)$.
- $\text{PKE.Enc}(pk, m_b, r)$ will not have the correct r and we will need to account for this in our analysis.
- For now, $\mathcal{D}$ does not know that, it only sees c^* and k^* and does its attack thing.
- At some point it outputs “yep” that c^* encapsulates k^* or “nope”.
- Or, it might crash and output nothing.
- In any event, we ignore its output.
- Rather, we check if it ever queried m_0 to the random oracles $H()$ or $K()$.
- If it did, we decide that c^* encrypts m_0 otherwise we decide that it encrypts m_1 .

BREAKING IND-CPA PKE III

$\mathcal{A}(\text{pk})$

```

1:  $\mathcal{C}, \mathcal{T}_H, \mathcal{T}_K, \mathcal{T}_c \leftarrow \emptyset, \emptyset, \emptyset, \emptyset$ 
2:  $m_0, m_1 \leftarrow \$ \{0, 1\}^\lambda, \{0, 1\}^\lambda$ 
3:  $b \leftarrow \$ \{0, 1\}; b' \leftarrow \mathcal{D}^{\mathcal{C}, \mathcal{D}, \mathcal{H}, \mathcal{K}}(\text{pk})$ 
4: if  $m_b \in \mathcal{T}_H.\text{keys} \vee m_b \in \mathcal{T}_K.\text{keys}$ 
5:   return  $b$ 
6: else return  $1 - b$ 
```

$\mathcal{H}(x)$

```

1: if  $x \notin \mathcal{T}_H.\text{keys}$ 
2:    $\mathcal{T}_H[x] \leftarrow \$ \{0, 1\}^\lambda$ 
3:  $c \leftarrow \text{PKE}.\text{Enc}(\text{pk}, x, \mathcal{T}_H[x])$ 
4:  $\mathcal{T}_c[c] \leftarrow x$ 
5: return  $\mathcal{T}_H[x]$ 
```

$\mathcal{C}()$

```

1: // IND-CPA Oracle
2:  $c^* \leftarrow \text{PKE}.\mathcal{C}(m_0, m_1)$ 
3:  $\mathcal{C} \leftarrow \mathcal{C} \cup \{c^*\}$ 
4:  $k^* \leftarrow \$ \{0, 1\}^\lambda$ 
5: return  $c^*, k^*$ 
```

$\mathcal{K}(x)$

```

1: if  $x \notin \mathcal{T}_K.\text{keys}$ 
2:    $\mathcal{T}_K[x] \leftarrow \$ \{0, 1\}^\lambda$ 
3:  $c \leftarrow \text{PKE}.\text{Enc}(\text{pk}, x, \mathcal{H}(x))$ 
4:  $\mathcal{T}_c[c] \leftarrow x$ 
5: return  $\mathcal{T}_K[x]$ 
```

$\text{Decap}(c)$

```

1: if  $c \in \mathcal{C}$  then
2:   return  $\perp$ 
3: if  $c \in \mathcal{T}_c.\text{keys}$ 
4:   return  $\mathcal{K}(\mathcal{T}_c[c])$ 
5: else return  $\perp$ 
6:
```

- Note that if $\mathcal{D}$ ever queries m_0 to either RO, it must have gotten it from somewhere.
- We have to account for the probability that it queried it by chance but that probability is low if the space from which we draw m_0 and m_1 is sufficiently large.
- If c^* encrypts m_1 then no information related to m_0 is given to $\mathcal{D}$.
- Thus, if $\mathcal{D}$ queried m_0 it must have gotten it from c^* . Nothing else depends on m_0 .

BREAKING IND-CPA PKE v

We also know that $\mathcal{D}$ will query m_0 if c^* encrypts it.

- $\mathcal{D}$ is a successful adversary in the IND-CCA KEM game, which requires it to touch m_0 at some point.
- It must check if k^* matches $K(m_0)$ to decide if k^* is the key encapsulated under c^* .
- In the random oracle model k^* is just a random string and it can only check by querying $K(m_0)$.
- Instead of querying $K(m_0)$, $\mathcal{D}$ might realise that we are cheating as $\text{PKE.Enc}(\text{pk}, m_0, H(m_0))$ will not match c^* , the randomness is not right.
- It might then get really angry, flip over tables, whatnot.
- It does not matter, at that point it already solved our problem: $\mathcal{D}$ queried $H(m_0)$.
- The moment it is able to detect that we are cheating is the moment we win.

Summary

If $\mathcal{D}$ is a successful adversary then it must either query $K(m_0)$ to check if k^* is correct or $H(m_0)$ to check if we are cheating. In either case we win: we've built $\mathcal{A}$ from $\mathcal{D}$. Now, since we assumed $\mathcal{A}$ does not exist, $\mathcal{D}$ does not exist.

FIN

IN THE RANDOM ORACLE MODEL THE
ADVERSARY SENDS US ITS
RANDOMNESS.

- [AOPPS17] Martin R. Albrecht, Emmanuela Orsini, Kenneth G. Paterson, Guy Peer, and Nigel P. Smart. **Tightly Secure Ring-LWE Based Key Encapsulation with Short Ciphertexts**. In: *ESORICS 2017, Part I*. Ed. by Simon N. Foley, Dieter Gollmann, and Einar Sneekkenes. Vol. 10492. LNCS. Springer, Cham, Sept. 2017, pp. 29–46. DOI: 10.1007/978-3-319-66402-6_4.
- [Den03] Alexander W. Dent. **A Designer's Guide to KEMs**. In: *9th IMA International Conference on Cryptography and Coding*. Ed. by Kenneth G. Paterson. Vol. 2898. LNCS. Springer, Berlin, Heidelberg, Dec. 2003, pp. 133–151. DOI: 10.1007/978-3-540-40974-8_12.
- [FO13] Eiichiro Fujisaki and Tatsuaki Okamoto. **Secure Integration of Asymmetric and Symmetric Encryption Schemes**. In: *Journal of Cryptology* 26.1 (Jan. 2013), pp. 80–101. DOI: 10.1007/s00145-011-9114-1.

- [HHK17] Dennis Hofheinz, Kathrin Hövelmanns, and Eike Kiltz. **A Modular Analysis of the Fujisaki-Okamoto Transformation**. In: *TCC 2017, Part I*. Ed. by Yael Kalai and Leonid Reyzin. Vol. 10677. LNCS. Springer, Cham, Nov. 2017, pp. 341–371. DOI: 10.1007/978-3-319-70500-2_12.